Design and Implementation of Digital Logic Circuits Using Verilog HDL

Anirudh Chaturvedi (SR 25850)
Microelectronics and VLSI Design, Dept. of ECE
Indian Institute of Science, Bangalore
Email: anirudhc@iisc.ac.in

Abstract

This report presents the design, simulation, synthesis, and FPGA implementation of digital logic circuits using Verilog HDL. The experiments include a 4-bit 4-to-1 multiplexer, a 4-bit carry look-ahead adder, and a parameterized synchronous mod-N up-down counter. Functional, pre-timing, and post-timing simulations are performed targeting the Xilinx Artix-7 FPGA. Resource utilization, timing performance, and power consumption are analyzed.

Index Terms

Verilog HDL, FPGA, Artix-7, Multiplexer, Carry Look-Ahead Adder, Counter

I. 4-bit 4-to-1 Multiplexer

A. Introduction

This experiment involves the design and verification of a 4-bit 4-to-1 multiplexer using Verilog HDL. The objective is to understand combinational logic modeling, simulation, and synthesis on FPGA.

B. Architecture Design

The multiplexer selects one of four 4-bit input vectors based on two select lines. A hand-drawn architecture illustrating the datapath and selection logic is shown in Fig. 1.

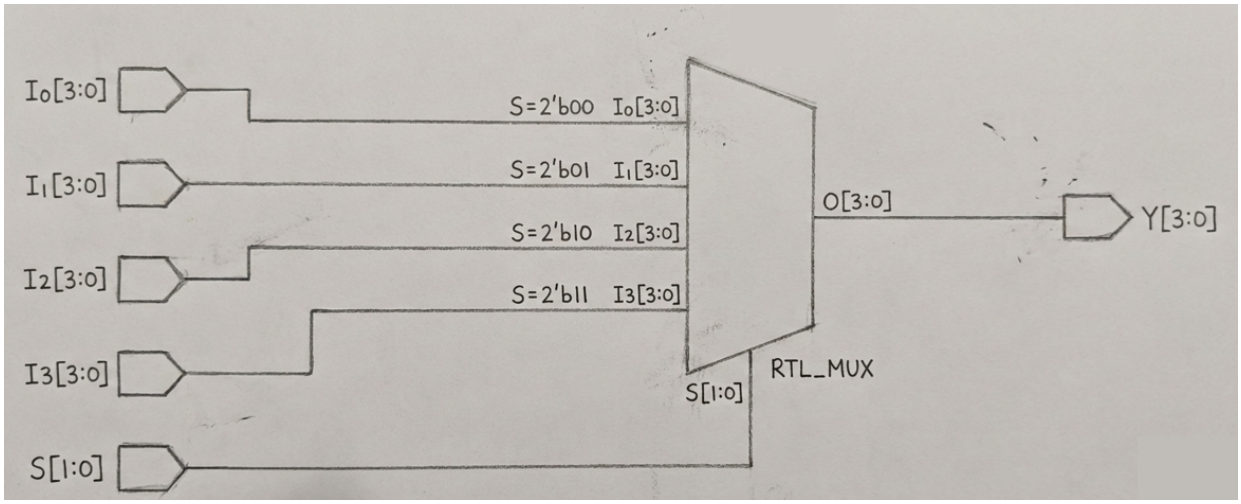


Fig. 1: Hand-drawn architecture of 4-bit 4-to-1 multiplexer

C. RTL Schematic

The RTL schematic generated by Vivado after synthesis is shown in Fig. 2.

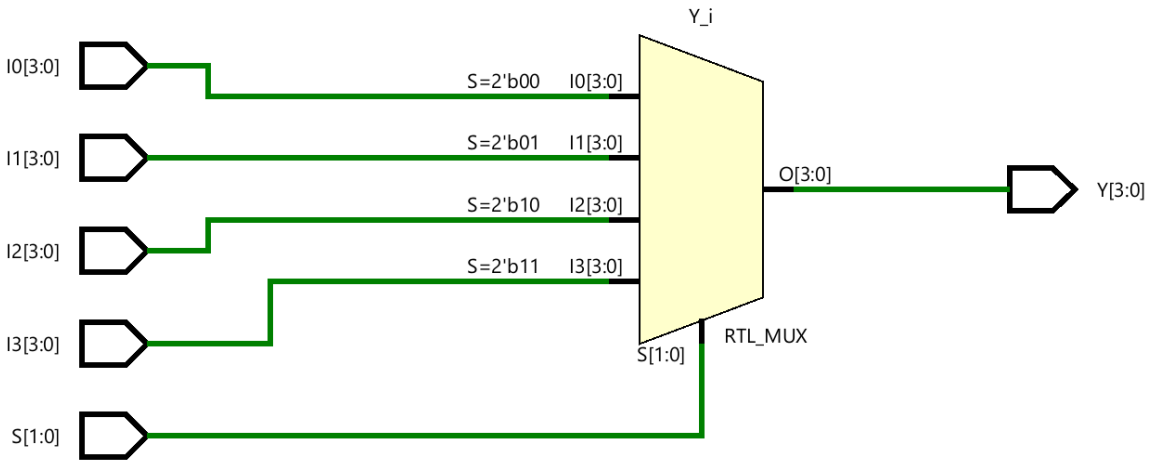


Fig. 2: RTL schematic generated by Vivado

D. Testbench Simulations

To comprehensively verify the design, simulations were carried out at various stages of the design flow. These include behavioral simulation, post-synthesis functional and timing simulations, and post-implementation functional and timing simulations. Each stage ensures correctness of functionality as well as timing closure.

1) Behavioral Simulation: Behavioral simulation was performed to verify the logical correctness of the design independent of synthesis and timing effects.

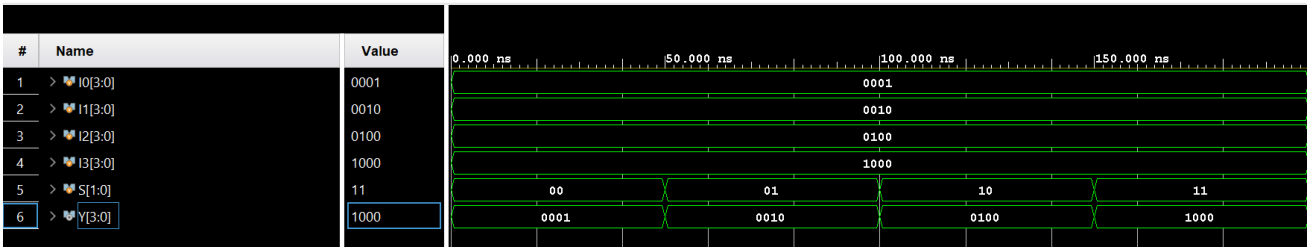


Fig. 3: Behavioral (RTL) simulation waveform of 4-bit multiplexer

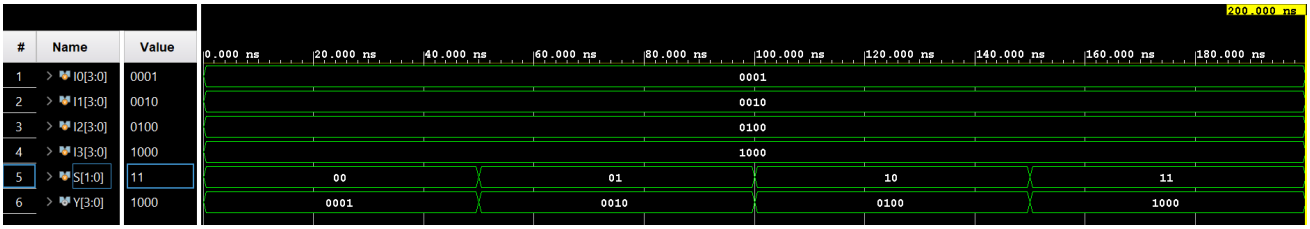


Fig. 4: Post-synthesis functional simulation waveform

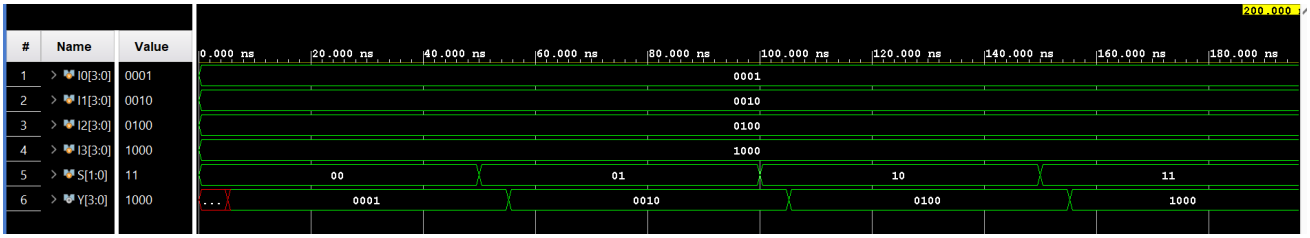


Fig. 5: Post-synthesis timing simulation waveform

2) Post-Synthesis Simulations: Post-synthesis simulations validate that the synthesized netlist preserves functional behavior and meets initial timing requirements.

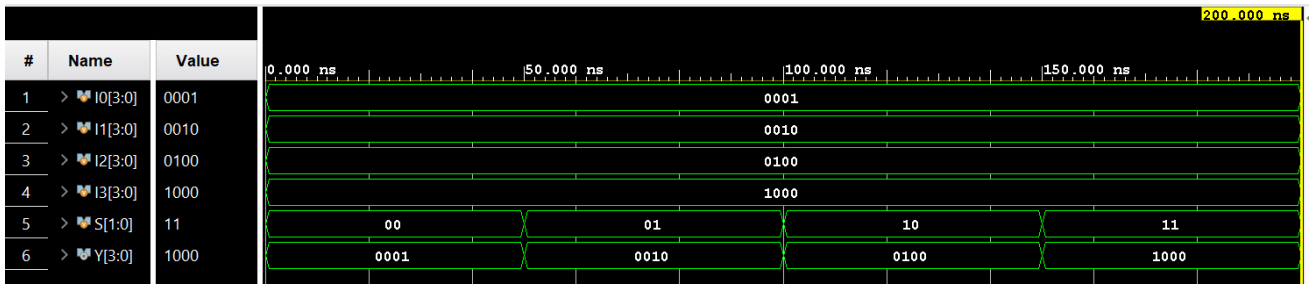


Fig. 6: Post-implementation functional simulation waveform

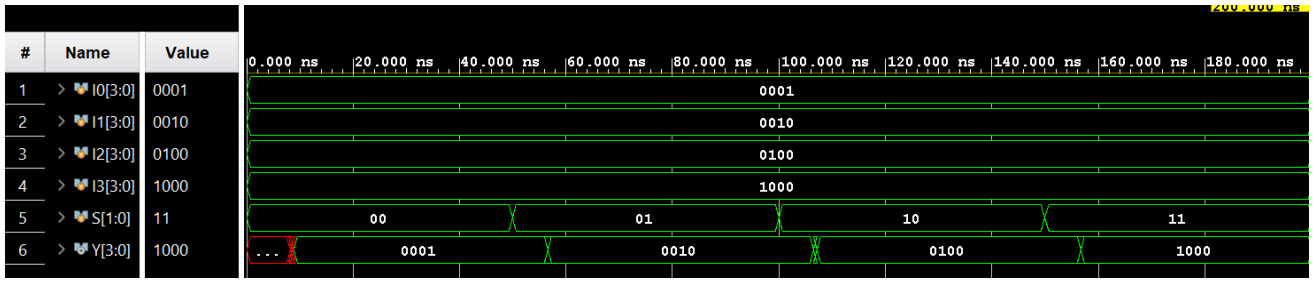


Fig. 7: Post-implementation timing simulation waveform

3) Post-Implementation Simulations: Post-implementation simulations incorporate routing delays and confirm that the design operates correctly under realistic timing conditions.

E. Resource Utilization

The resource utilization of the design was obtained from the Vivado synthesis report. The design utilizes a small number of LUTs and does not require any flip-flops, indicating efficient hardware implementation for a purely combinational circuit.

Utilization Post-Synthesis | **Post-Implementation**

Graph | **Table**

Resource	Utilization	Available	Utilization %
LUT	4	20800	0.02
IO	22	106	20.75

Fig. 8: Resource utilization report generated by Vivado

F. Power Consumption

Power analysis was performed using Vivado's power estimation tool. Due to the combinational nature of the design, both static and dynamic power consumption are negligible.

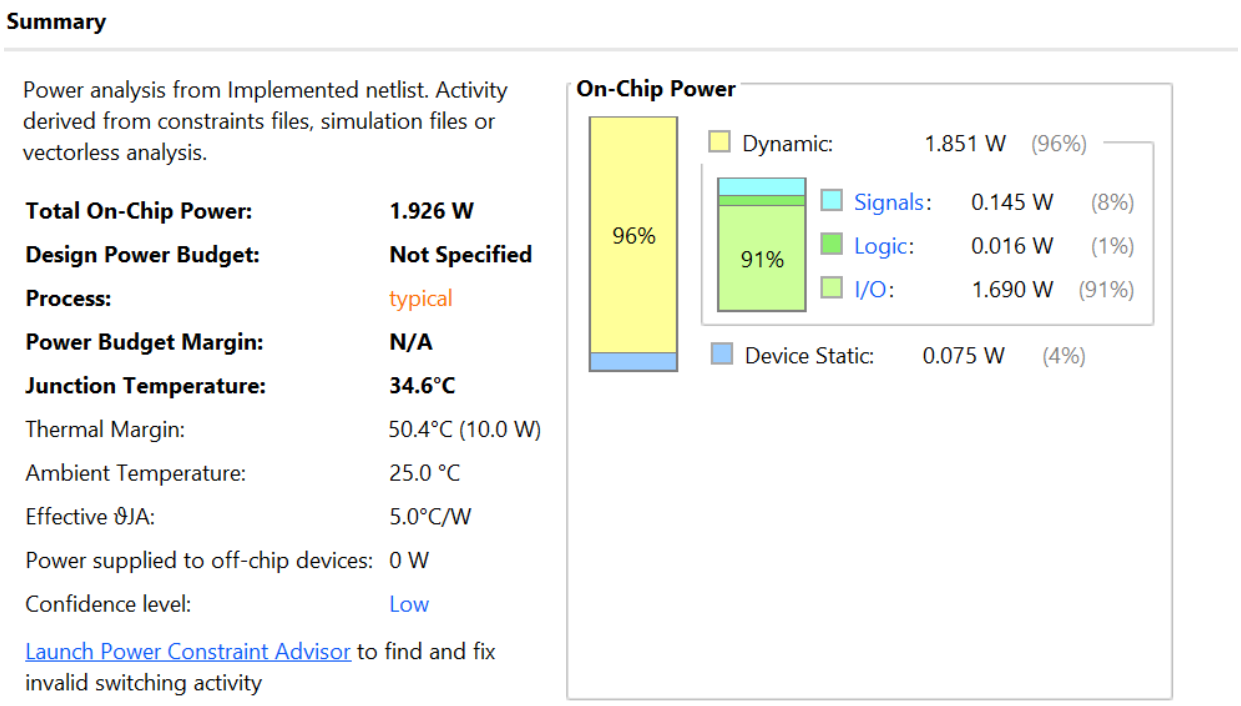


Fig. 9: Power estimation report generated by Vivado

G. Conclusion

The 4-bit 4-to-1 multiplexer was successfully designed, simulated, and synthesized with correct functional behavior and minimal resource usage.

II. 4-bit Carry Look-Ahead Adder

A. Introduction

This experiment focuses on implementing a 4-bit carry look-ahead adder (CLA) using continuous assignments in Verilog HDL. The objective is to reduce carry propagation delay by computing carry signals in parallel, thereby achieving faster addition compared to ripple-carry adders.

B. Architecture Design

The CLA architecture employs generate (G) and propagate (P) signals to compute intermediate carries in parallel. A hand-drawn flop-level schematic illustrating the carry generation and summation datapath is shown in Fig. 10.

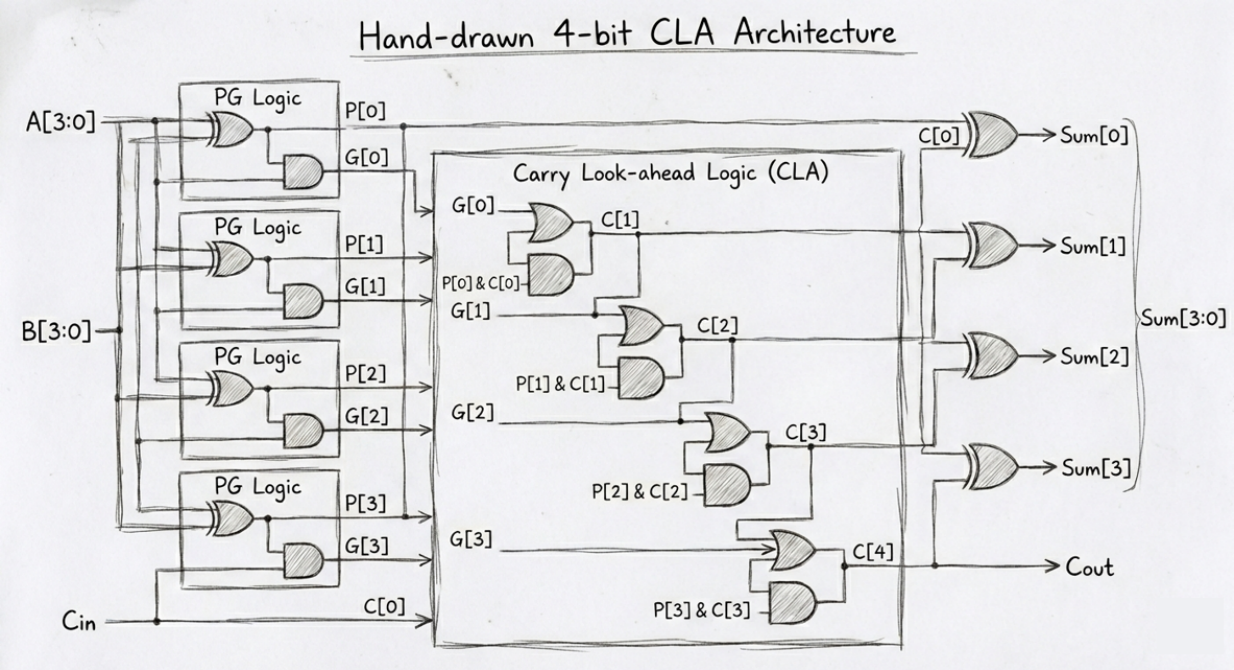


Fig. 10: Hand-drawn architecture of 4-bit carry look-ahead adder

C. RTL Schematic

The RTL schematic generated by Vivado after synthesis is shown in Fig. 11.

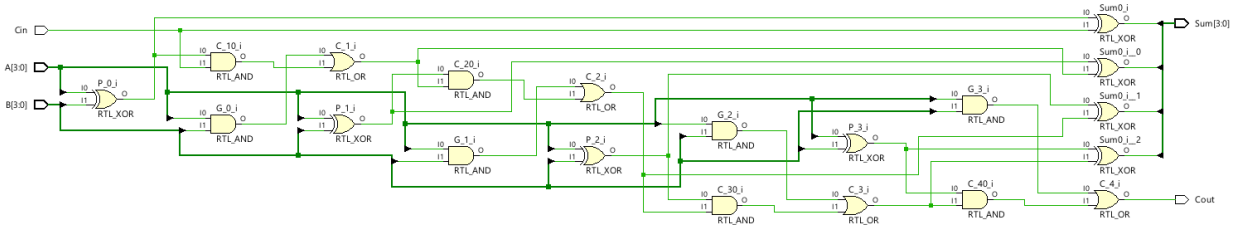


Fig. 11: RTL schematic of 4-bit carry look-ahead adder

D. Testbench Simulations

To verify both functional correctness and timing behavior, simulations were performed at multiple stages of the design flow, including behavioral, post-synthesis, and post-implementation simulations.

1) Behavioral Simulation: Behavioral simulation was carried out to validate correct sum and carry outputs for all input combinations without considering synthesis or routing delays.

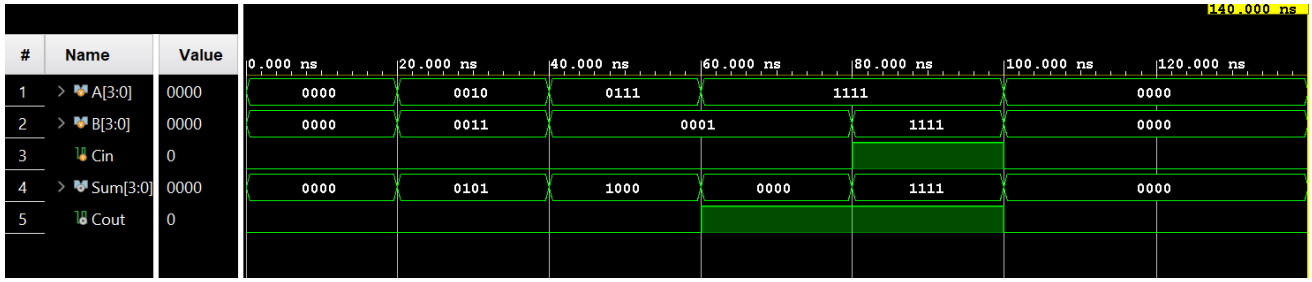


Fig. 12: Behavioral (RTL) simulation waveform of 4-bit CLA

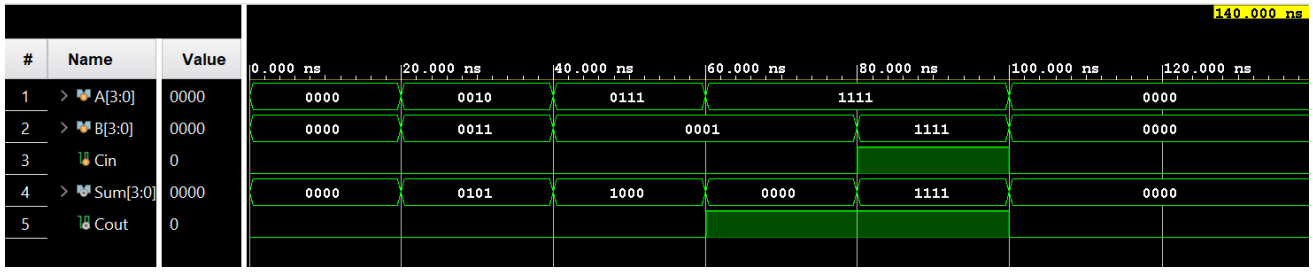


Fig. 13: Post-synthesis functional simulation waveform

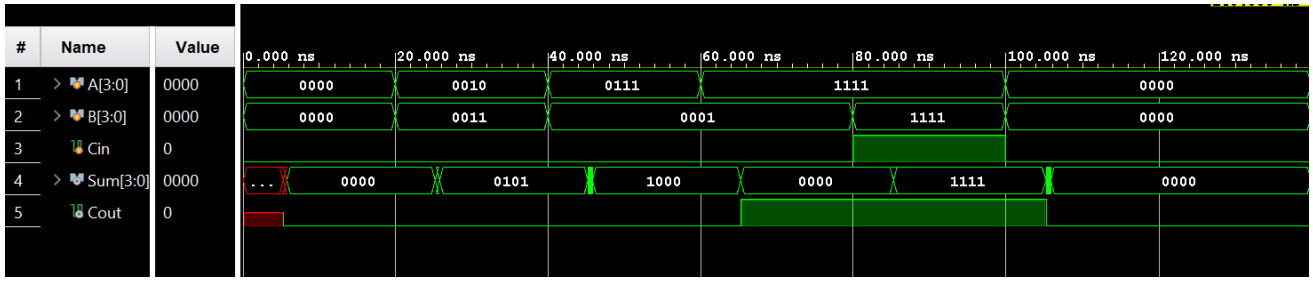


Fig. 14: Post-synthesis timing simulation waveform

2) Post-Synthesis Simulations: Post-synthesis simulations confirm that the synthesized netlist preserves functional behavior while introducing gate-level delays.

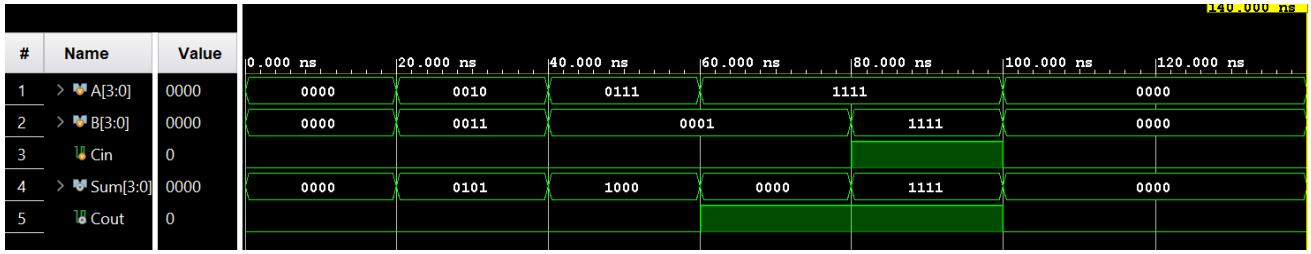


Fig. 15: Post-implementation functional simulation waveform

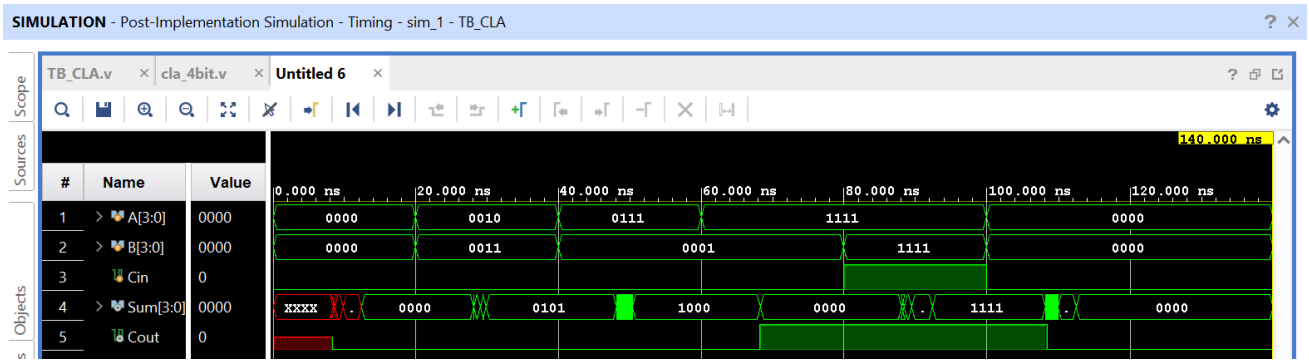


Fig. 16: Post-implementation timing simulation waveform

3) Post-Implementation Simulations: Post-implementation simulations include routing delays and verify correct operation under realistic timing conditions.

E. Resource Utilization

Resource utilization was obtained from the Vivado implementation report. Due to the parallel carry computation logic, the CLA consumes more LUTs compared to a ripple-carry adder, while requiring no flip-flops.

Utilization Post-Synthesis | **Post-Implementation**

Graph | **Table**

Resource	Utilization	Available	Utilization %
LUT	4	20800	0.02
IO	14	106	13.21

Fig. 17: Resource utilization report for 4-bit CLA

F. Power Consumption

Power analysis was performed using Vivado's power estimation tool. The CLA exhibits slightly higher dynamic power consumption due to increased combinational switching activity associated with parallel carry logic.

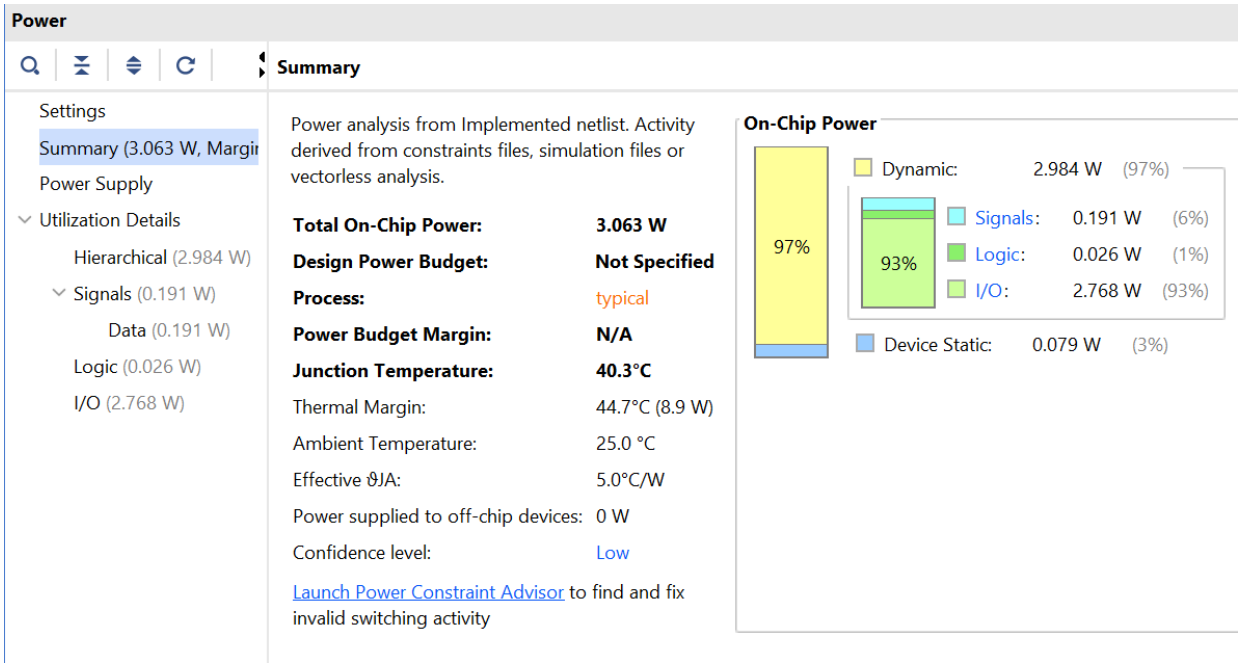


Fig. 18: Power estimation report for 4-bit CLA

G. Conclusion

The 4-bit carry look-ahead adder was successfully designed, simulated, and implemented. The design demonstrates improved speed performance at the cost of modest increases in resource usage and power consumption, highlighting the classic trade-off between speed and hardware complexity.

III. Parameterized Synchronous Mod-N Up-Down Counter

A. Introduction

This experiment involves the progressive design of a synchronous counter system. Starting from a basic mod-10 counter, additional functionalities and parameterization are introduced. The design is finally implemented on the Basys3 FPGA with proper frequency control and LED visualization.

B. Design Evolution

1) Part (a): Basic Synchronous Mod-10 Counter: A 4-bit synchronous up counter with active-low reset was designed. The counter increments on every rising clock edge and rolls over after reaching decimal 9.

2) Part (b): Extended Counter Functionalities: The design was extended to support multiple operating modes including hold, count up, count down, up-down counting, load, and local synchronous reset. All operations are controlled within a single synchronous process.

3) Part (c): Parameterized Mod-N Counter: The counter was generalized to operate as a mod-N counter with $N \leq 16$. The value of N determines rollover and underflow conditions, allowing runtime configurability without altering the hardware structure.

4) Part (d): Clock Divider Based FPGA Visualization: To visualize the counter output on the Basys3 LEDs, the 100 MHz system clock was reduced to approximately 1 Hz using a basic clock divider. The divided clock directly drives the counter, enabling human-observable counting behavior. While functional, this approach introduces an internally generated clock and is not ideal for FPGA timing.

5) Part (e): Clock Enable Based Synchronous Design: To address the limitations of internally generated clocks, the final design employs a clock-enable based approach. A 1 Hz enable pulse controls state updates while all sequential logic remains driven by the primary 100 MHz clock. This approach ensures safe clocking, reliable timing closure, and efficient power utilization.

A hand-drawn flop-level schematic illustrating the final clock-enable based architecture is shown in Fig. 19.

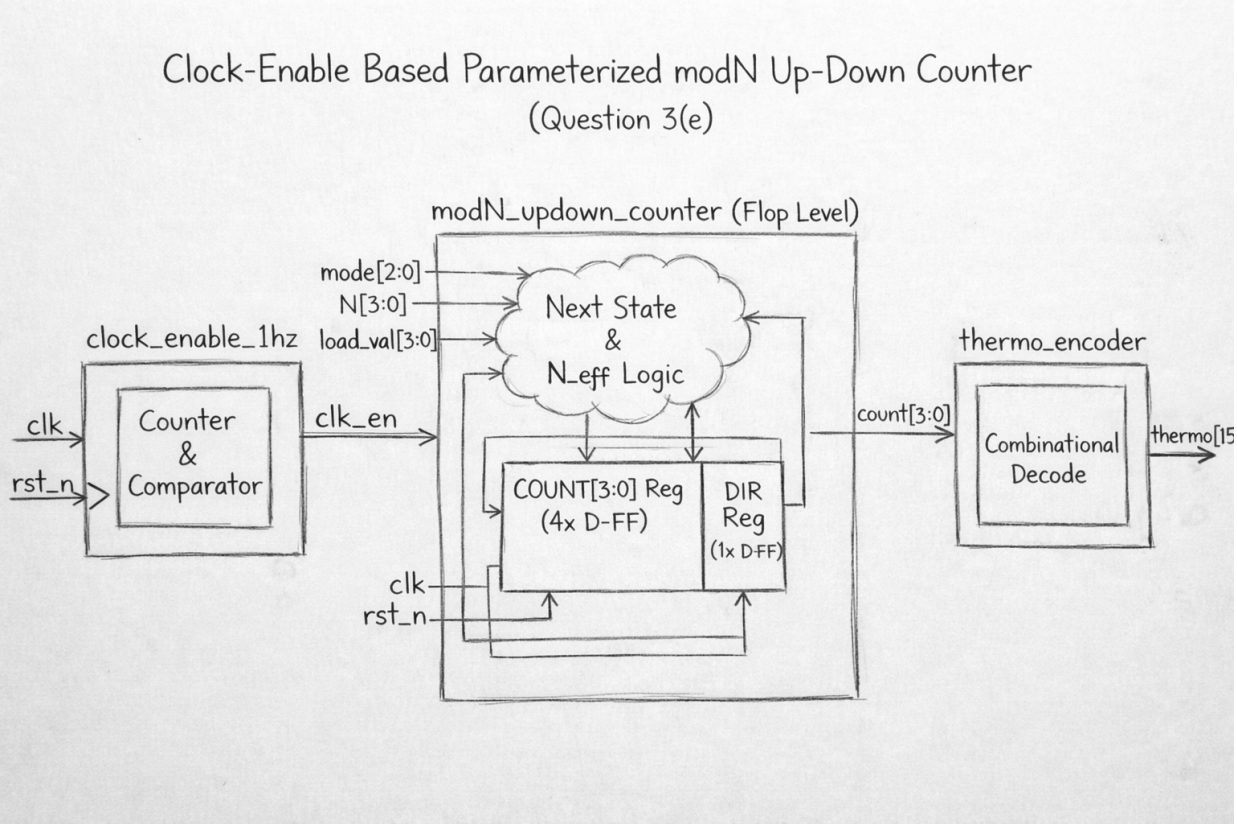


Fig. 19: Hand-drawn architecture of clock-enable based mod-N up-down counter

C. RTL Schematic

The RTL schematic of the final based design generated by Vivado is shown in Fig. 20. The schematic confirms correct inference of registers, control logic, and clock-enable circuitry.

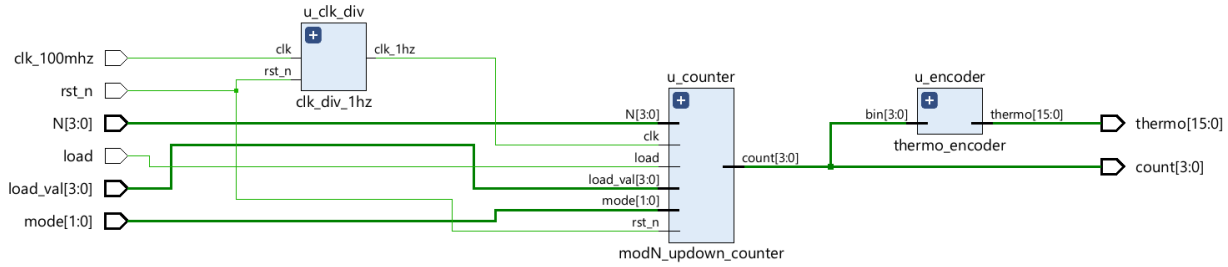


Fig. 20: RTL schematic of clock-dicider based mod-N counter

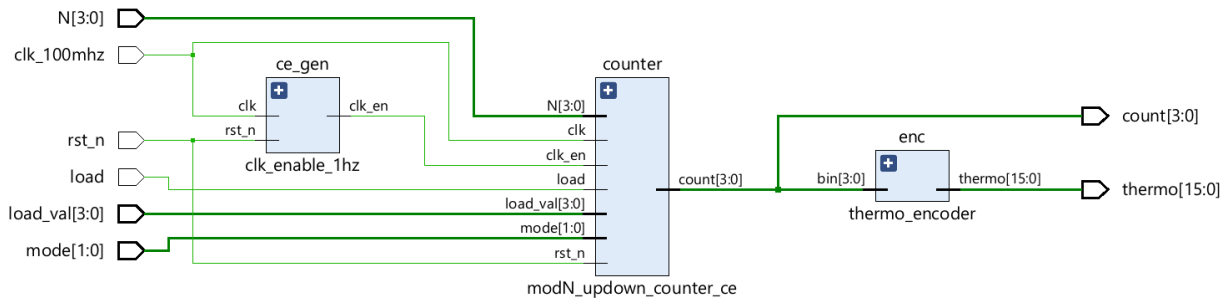


Fig. 21: RTL schematic of clock-enable based mod-N counter

D. Testbench Simulations

Simulations were performed to verify functional correctness and timing behavior of the final design (clock-enable).

1) Behavioral Simulation: Behavioral simulation verifies correct operation of all counter modes without considering logic or routing delays.

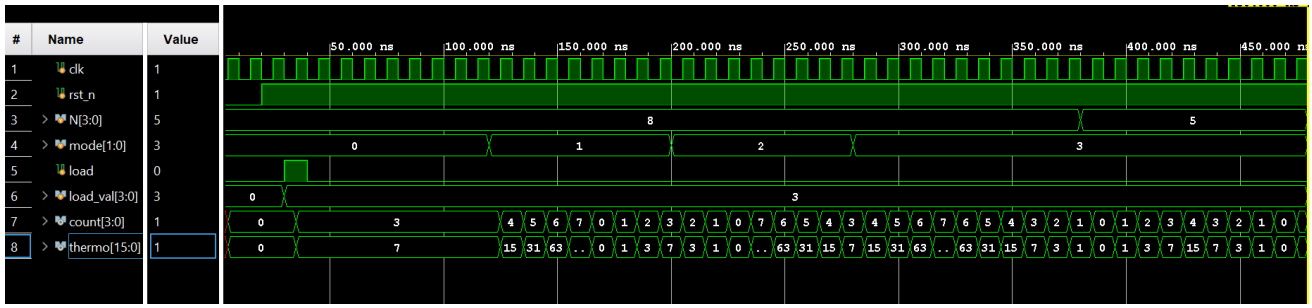


Fig. 22: Behavioral simulation waveform of mod-N counter

2) Post-Synthesis and Post-Implementation Simulations: Post-synthesis and post-implementation simulations confirm that the synthesized and placed designs preserve functional correctness while accounting for logic and routing delays.

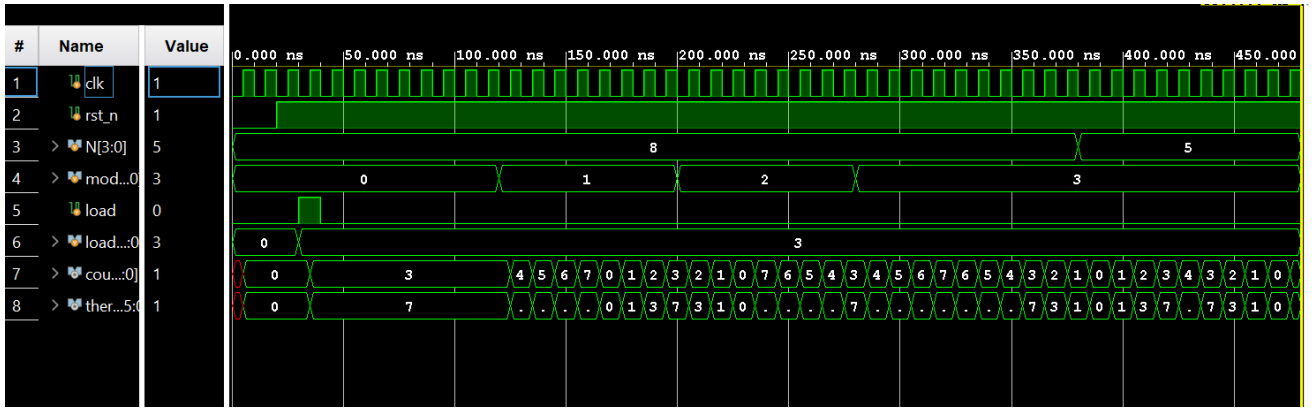


Fig. 23: Post-synthesis functional simulation

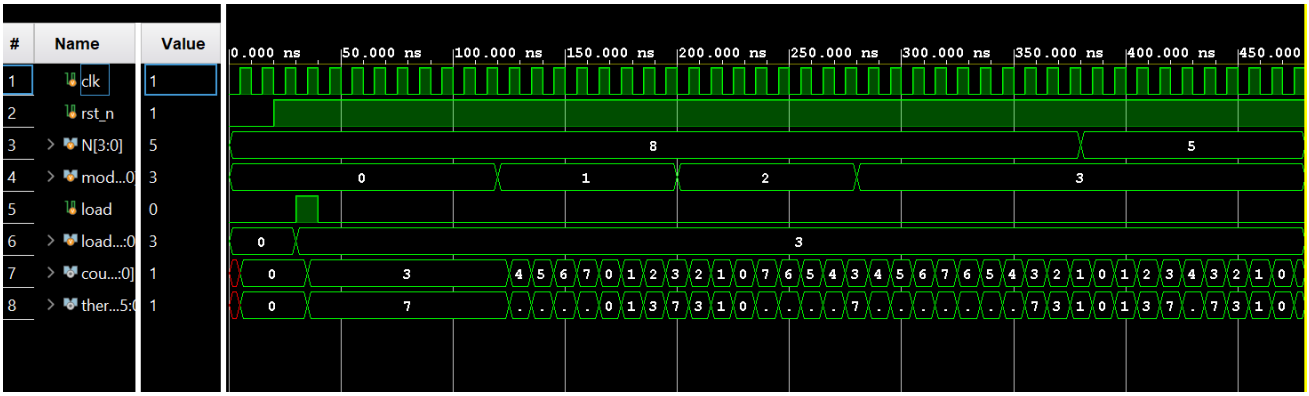


Fig. 24: Post-synthesis timing simulation

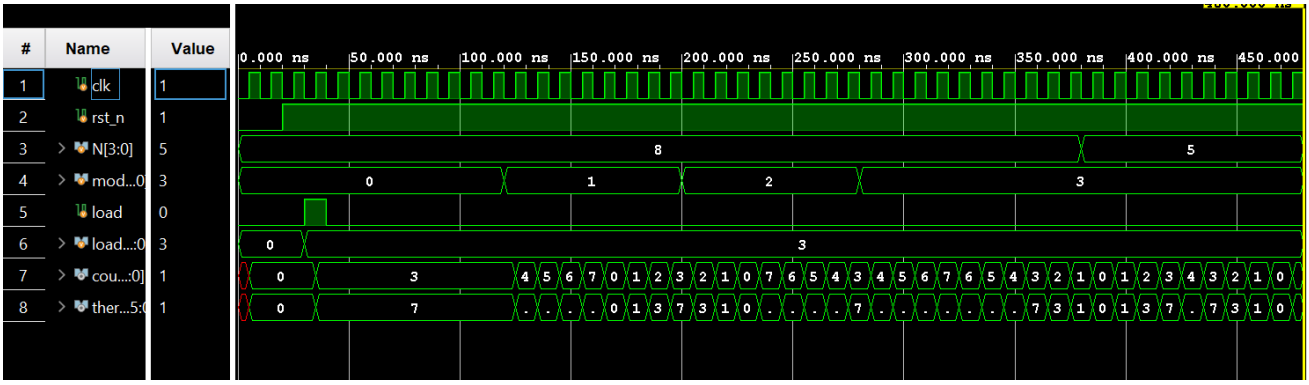


Fig. 25: Post-implementation functional simulation

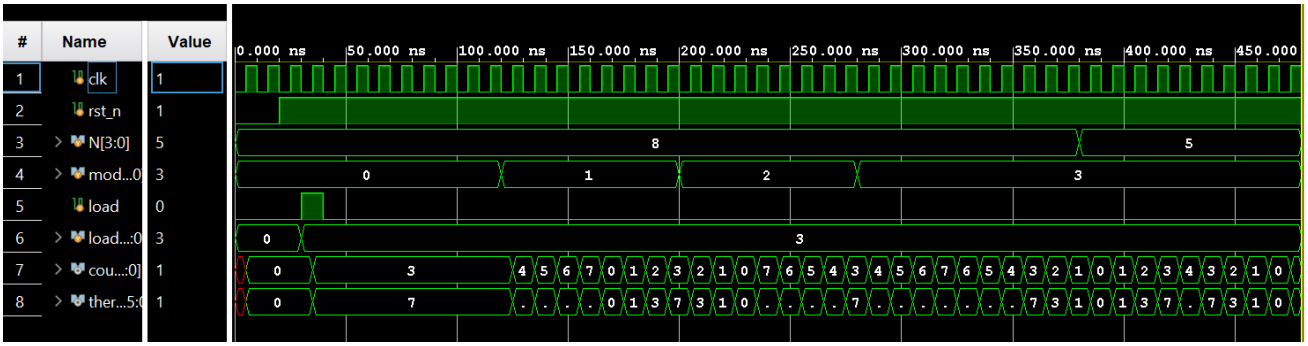


Fig. 26: Post-implementation timing simulation

E. Resource Utilization

The final clock-enable based design utilizes a moderate number of LUTs and flip-flops, remaining well within the available resources of the Artix-7 FPGA.

Utilization Post-Synthesis | **Post-Implementation**

Graph | **Table**

Resource	Utilization	Available	Utilization %
LUT	40	20800	0.19
FF	33	41600	0.08
IO	29	106	27.36
BUFG	1	32	3.13

Fig. 27: Resource utilization report for clock-enable based design

F. Timing Summary

The post-route timing report confirms that all timing constraints are met. The use of clock-enable logic eliminates issues associated with internally generated clocks and ensures robust timing closure.

Design Timing Summary

Setup		Hold		Pulse Width	
Worst Negative Slack (WNS):	4.960 ns	Worst Hold Slack (WHS):	0.261 ns	Worst Pulse Width Slack (WPWS):	4.500 ns
Total Negative Slack (TNS):	0.000 ns	Total Hold Slack (THS):	0.000 ns	Total Pulse Width Negative Slack (TPWS):	0.000 ns
Number of Failing Endpoints:	0	Number of Failing Endpoints:	0	Number of Failing Endpoints:	0
Total Number of Endpoints:	64	Total Number of Endpoints:	64	Total Number of Endpoints:	34

All user specified timing constraints are met.

Fig. 28: Post-route timing summary

G. Power Consumption

Power analysis indicates low dynamic and static power consumption. The clock-enable based approach minimizes unnecessary switching activity and improves overall power efficiency.

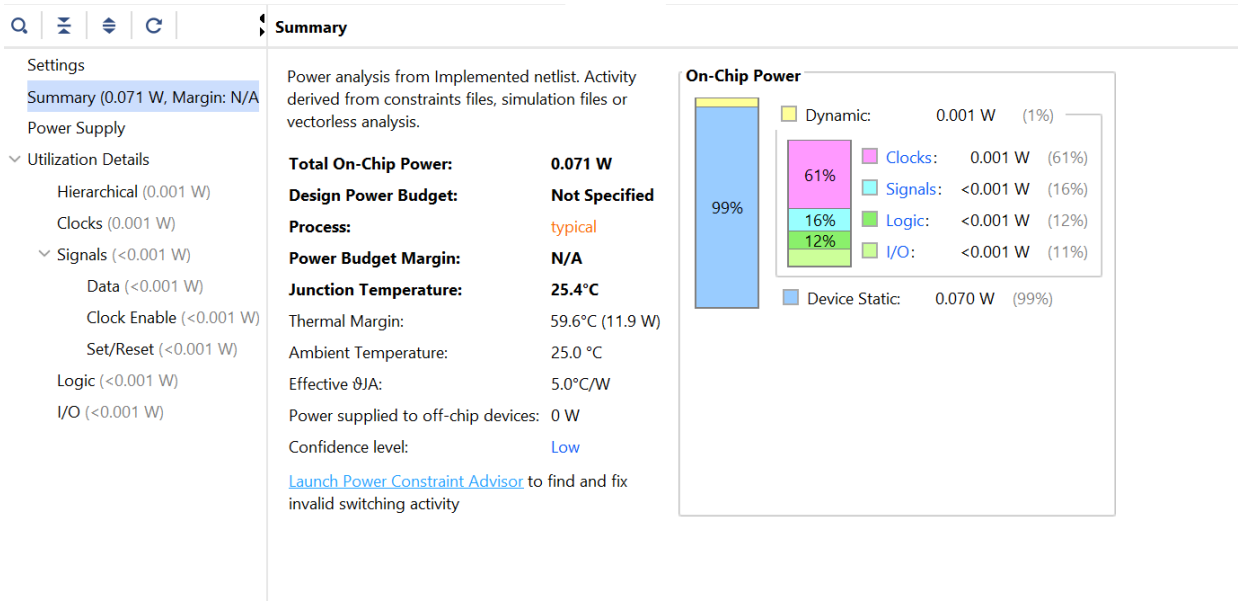


Fig. 29: Power estimation report

H. Conclusion

The design demonstrates a systematic progression from a basic synchronous counter to a robust FPGA-ready implementation. While a clock-divider based approach enables basic visualization, the clock-enable based design provides a timing-safe, power-efficient, and scalable solution suitable for FPGA implementation.

REFERENCES

[1] S. Palnitkar, *Verilog HDL: A Guide to Digital Design and Synthesis*, Prentice Hall, 2003.
[2] S. Brown and Z. Vranesic, *Fundamentals of Digital Logic with Verilog Design*, McGraw-Hill, 2014.
[3] Xilinx Inc., *7 Series FPGAs Data Sheet: Overview*, 2022.
[4] IEEE Standard for Verilog Hardware Description Language, IEEE Std 1364-2005.